Learny · Gamificación e infraestructura: análisis completo

Base analizada: Learny `learny-v99` (esquema de datos `version: 4`, metas `learning.v: 1`). El análisis original se hizo sobre la `learny-v98`; esta versión lo revisa contra la v99 y las conclusiones no cambian (ver «Revisión contra la v99» abajo). Este documento reúne (1) qué mecánicas de gamificación ya existen y cuáles faltan, (2) las restricciones del código actual, (3) un modelo de datos propuesto y (4) las condiciones de Firebase (Spark) y Cloudflare Workers que hay que tener en cuenta en las siguientes fases. Todo está ordenado de lo más esencial a lo más mínimo.

Aviso sobre las fuentes: lo que describe el código sale de leer los archivos del proyecto. Lo de Firebase y Cloudflare sale de la revisión de páginas oficiales que pegaste; los límites cambian, así que deben reconfirmarse antes de implementar.

Revisión contra la v99: la v99 solo corrigió regresiones y pruebas (`settings.newPerDay`, manejadores de `data-empty="more-new"`, `data-task-go` y `data-leech-open`, foco del resultado de la IA y la clase `w-edge` de la bienvenida). No tocó la racha, el quiz, `state.reviews`, `PRO_FEATURES` ni Firebase. Se comprobó en el código que siguen valiendo: `calcStreak()` solo mira sesiones, el quiz guarda 20 resultados en `learny.quizzes`, `MAX_REVIEWS = 5000`, `PRO_FEATURES` lista `xp` como no listo y los diez eventos citados en la sección 2 existen.

1. Lo esencial (resumen ejecutivo)

Conclusiones clave:

1. Ni XP, ni monedas, ni insignias, ni escudos tienen dónde guardarse. Hace falta un contenedor nuevo (propuesta: `state.game`).
2. La racha actual no cuenta repasos ni quizzes, que es justo lo que pide la mecánica.
3. Todo es local y editable: XP y monedas no sirven para ligas, rankings ni duelos sin servidor.
4. Para el servidor, Spark no incluye Cloud Functions; el cobro iría en Cloudflare Workers, con el límite de 10 ms de CPU como único riesgo técnico real.

Mecánica	Estado	Qué hay hoy
Racha de estudio	Parcial	`calcStreak()` cuenta solo bloques Pomodoro
Racha flexible	Parcial	«Día de gracia» automático; no hay escudo
XP por dominio	Falta	No existe el concepto
Niveles y rangos	Parcial	«Práctica nivel N» por materia, sin XP ni títulos
Barras de progreso	Parcial	Hay porcentajes y gráficos; no «% dominado»
Insignias	Parcial	Un único aviso «Buen resultado», sin registro
Misiones diarias	Parcial	3 misiones fijas, sin recompensa
Moneda virtual	Falta	Nada, y no hay nada que canjear

2. Reglas que no se deben romper

- No cambiar SM-2, el modelo de tarjetas ni `SCHEMA` (sigue en 4).
- No cambiar Pomodoro, Feynman ni estadísticas existentes.
- La gamificación debe leer eventos, no modificar el repaso. Eventos disponibles: `learny:rated`, `learny:pomodone`, `learny:examend`, `learny:examstart`, `learny:timer`, `learny:page`, `learny:render`, `learny:reset`, `learny:pro`, `learny:profile`.
- Regla de proporción de `CONTINUIDAD.md`: cada novedad ornamental (Lumi, temas, animaciones) exige antes una mejora del núcleo de estudio. La gamificación es motivación, no núcleo; conviene emparejarla con el contador unificado de actividad (que además corrige la racha).

- Premiar estudio real (recuerdo activo, quizzes, repasos espaciados), no tiempo pasivo ni solo abrir la app.
- No pay-to-win: Pro no debe dar más XP ni ventajas en rankings.
- Rachas no demasiado punitivas; competición opcional y en grupos pequeños.
- Accesibilidad: anuncios `aria-live` para subidas de nivel y logros, foco correcto y respeto del movimiento reducido (sistema y Learny).
- Traducción: cada texto nuevo necesita entradas en `EX` / `RULES` de `js/18-idioma.js`; el texto del usuario va en `translate="no"`.
- Un solo mensaje proactivo de Lumi por visita; si Lumi reacciona a niveles o insignias, debe respetarlo.

3. Detalle por mecánica

3.1 Racha de estudio (parcial)

Existe:

- `calcStreak()` / `calcStreakRaw()` en `js/05-temporizador.js`: días consecutivos con al menos una sesión no abandonada en `state.sessions`.
- Se muestra en «Más sobre hoy» (`#current-streak`) y en el texto de meta (`Racha: N días`).
- Lumi la menciona con racha ≥ 2 (`js/16-lumi.js`).
- Reforzar muestra «día de gracia usado/disponible».

Falta:

- Contar también repasos de tarjetas y quizzes completados (hoy solo cuentan bloques Pomodoro).
- El quiz (`js/21-quiz.js`) no deja registro en `state`: solo guarda los últimos 20 resultados en `learny.quizzes`. Para que cuente hay que decidir cómo registrarlo sin violar «un quiz no cambia estadísticas».
- Formato visual tipo «12 días estudiando» (la app usa iconos Material Symbols, no emojis).

3.2 Racha flexible (parcial)

Existe: un día de gracia automático: si faltas un día pero estudiaste el anterior, la racha no se rompe. Es una regla interna, no una elección del usuario.

Falta:

- El escudo como objeto visible: cómo se gana, cuántos se acumulan, cómo se usa.
- La sesión de rescate de 3 minutos para conservar la racha.
- Decidir si el día de gracia actual se mantiene, se sustituye o convive con el escudo.

3.3 XP por dominio (falta)

No existe nada. Hay que definir:

- Qué acciones dan XP (propuesta: respuesta «Fácil», tema ≥ 80 %, quiz aprobado, repaso completado) y cuánto.
- Que no se premien minutos pasados.
- Dónde se guarda el total.

Advertencia técnica: `state.reviews` guarda como máximo 5 000 repasos (`MAX_REVIEWS`) y se recorta, por lo que no puede ser la fuente del XP total. Hace falta un contador propio y acumulativo.

3.4 Niveles y rangos (parcial)

Existe: en Reforzar → «Más datos», `Práctica nivel ${1 + floor(sqrt(repasos))}` por materia (`extras.js`, `renderInsights`).

Falta:

- Nivel global y por materia derivados de XP.
- Títulos visibles (por ejemplo Aprendiz → Estratega → Maestro del examen).
- Curva de XP por nivel.

3.5 Barras de progreso (parcial)

Existe: minutos de foco por día, previsión de 7 días, retención real, «% sin fallar» por materia, progreso por tema (modo autodidacta) y etapas hechas por meta.

Falta:

- «% dominado» por materia, unidad, mazo o examen: no existe una definición de «dominado».
- Barras horizontales (las únicas barras actuales son verticales).
- Cuidar la redacción prudente del proyecto: autoevaluación no equivale a dominio comprobado.

3.6 Insignias (parcial)

Existe: `badges-list` muestra «Buen resultado en X» con 30 repasos de una materia y al menos 80 % sin fallar. Se recalcula; no se guarda nada.

Falta:

- Catálogo (primera semana, 100 tarjetas repasadas, simulacro aprobado, 7 días de repaso…).
- Registro de cuándo se ganó cada una.
- Un lugar donde verlas y un aviso al ganarlas.

3.7 Misiones diarias (parcial)

Existe: 3 misiones fijas en `extras.js` (`renderMissions`): repasar 10 tarjetas, completar un Pomodoro, explicar un tema con Feynman. Contador `#mission-count`.

Falta:

- Que cambien cada día y tengan variedad.
- Misiones con quiz («aprueba un mini-quiz»).
- Recompensa asociada (XP o moneda).
- Misiones semanales.

Nota: «Mostrar las misiones» es un ajuste (`showMissions`).

3.8 Moneda virtual (falta)

No existe. Además no hay nada que canjear: ya hay 7 opciones de tema (automático, oscuro, negro, claro, sepia, alto contraste, cromo), 5 acentos más personalizado y 8 apariencias de Lumi, pero todas están libres.

Falta decidir: qué se compra, a qué precio, y cómo impedir pay-to-win. Solo recompensas cosméticas, sin ventajas académicas de pago.

4. Faltantes transversales

1. **Contenedor de datos.** Siguiendo el patrón de `state.learning` (versión propia, sin subir `SCHEMA`): `state.game` con saneado en `hydrate`, validación al importar (`validateBackup`), vista previa de importación, copias, «Borrar todo» y sincronización entre pestañas (`adoptRemoteState`).
2. **Quiz fuera de** `state` (ver 3.1). Además es función Pro.
3. **Pro.** `PRO_FEATURES` lista `xp` («Gamificación: XP, niveles y medallas») como próximamente. Hay que decidir qué parte es gratis; tu propio análisis advierte contra pay-to-win.
4. **Anti-trampa.** Todo vive en `localStorage` y es editable. XP y monedas locales no sirven para ligas, rankings, duelos ni rachas compartidas.
5. **Funciones sociales** (ligas semanales, duelos de quiz, rachas compartidas, misiones de amigos, rankings por materia, torneos, sala de estudio virtual): dependen de grupos y servidor, que aún no existen (hoy solo hay cuentas y perfil).

6 **Traducción y accesibilidad** (ver sección 2).

7 **Lumi.** Puede reaccionar a niveles e insignias respetando sus reglas de frecuencia.

8 **Demo.** `?demo` no debe escribir nunca en una cuenta real ni en claves reales.

5. Modelo propuesto `state.game` (a decidir antes de programar)

Borrador, no definitivo:

```
game: {
  v: 1,
  xp: 0,                 // total acumulado, contador propio
  xpBySubject: { },      // XP por materia
  coins: 0,              // solo si se aprueba la moneda
  shields: 0,            // escudos de racha disponibles
  streakDays: [ 'AAAA-MM-DD' ],   // días con actividad válida (tope y recorte)
  badges: [ { id, date } ],
  daily: { date, missions: [ { id, done, rewarded } ] },
  log: [ { date, kind, amount } ]  // historial corto con tope
}
```

Puntos de diseño:

- Topes de tamaño y recorte, como `planLog` y `learning`.
- Idempotencia: un evento repetido (doble pulsación, recarga, otra pestaña) no debe sumar dos veces. Usar ids de sesión/repaso, como hace Lumi con `celebrated`.
- Si no es objeto o es de versión más nueva, comportarse como `learning` (ignorar o solo lectura con aviso).
- Fuente única de «actividad del día» (repaso, bloque o quiz) para racha, misiones y XP.

6. Ideas adicionales de la propuesta (para fases posteriores)

- Mapa de aventura por materia con nodos y jefes de examen (simulacro como «jefe»).
- Sistema de maestría (80 % de aciertos para avanzar) y nivel de confianza por tema (débil, en progreso, dominado).
- Cuenta regresiva de examen que intensifica retos.
- Cofres de recompensa al terminar sesión.
- «Modo Supervivencia», «Cazador de vacíos», «Racha inteligente», «Guardián del examen», «Batalla de materias» (solo contra uno mismo), «Cápsulas de repaso» de 5 minutos, «Reto de precisión».
- «Cazador de vacíos» podría usar la lista de tarjetas que se resisten (`leechCards`) y `weakSubject()` ya existentes, sin IA.

7. Condiciones de Firebase (plan Spark) a tener en cuenta

Estado en el proyecto: cuentas, perfil y `isPro()` desde Firestore ya están hechos hasta el Bloque 6. El Bloque 7 (activación de Pro por servidor) está pendiente. Las reglas actuales (`firebase/firestore.rules`) permiten crear solo con plan `free` y actualizar únicamente `displayName` y `updatedAt`. Guardar datos de juego en la nube exigiría reglas nuevas y revisión de seguridad.

Lo bueno (según la revisión):

- Spark no pide método de pago.
- Firestore gratis: 1 GiB almacenado, 50 000 lecturas/día, 20 000 escrituras/día, 20 000 borrados/día y 10 GiB/mes de salida.
- Authentication: 50 000 usuarios activos al mes.

Qué revisar:

1 Límites diarios de Firestore: al superarlos las operaciones fallan hasta el día siguiente (no se cobra). El perfil se lee en cada inicio de sesión.

2 Correos de Authentication: 150 de recuperación de contraseña al día y 1 000 de verificación al día. Enlace por correo sin contraseña: 5 al día en Spark, no usarlo.

3 Registros por IP: 100 cuentas nuevas por hora desde la misma IP (relevante en un aula con una sola red).

4 Usuarios activos diarios: la página lista «Tier 1 = 3 000/día» para Spark; no quedó claro a qué proveedores aplica. Confirmarlo en la consola.

5 Spark no incluye Cloud Functions, Cloud Storage, App Hosting ni Cloud Secrets Manager. Por eso el cobro automático iría en Cloudflare.

6 Cuota compartida: lo que escriba el Worker en Firestore cuenta contra las 20 000 escrituras diarias.

Impacto en gamificación:

- XP, monedas e insignias sincronizados serían escrituras nuevas. Agrupar por sesión o por día, no por repaso.
- No usar escuchas en tiempo real innecesarias: hoy el consumo es bajo (una lectura de perfil por inicio de sesión, sin `onSnapshot`).
- Rankings y ligas multiplican lecturas: diseñarlos con consultas acotadas.

8. Condiciones de Cloudflare Workers (plan gratuito)

Lo bueno: 100 000 solicitudes al día (se reinician a medianoche UTC) y hasta 100 Workers por cuenta.

Qué revisar:

1 **10 ms de CPU por solicitud.** Es el límite más importante. Para escribir en Firestore, el Worker debe firmar un token con la clave de servicio (RSA). Suele ser rápido pero puede rozar el límite: probar con un pago de prueba antes de confiar. Si falla, la alternativa es el plan de pago de Workers, el único costo posible.

2 50 subrequests por invocación: sobra (se necesitan 2 o 3: token y escritura).

3 Si se superan las 100 000 al día, Cloudflare devuelve el error 1027 y los avisos de pago pueden perderse. Preguntar al proveedor si reintenta los avisos fallidos.

4 Secretos: hasta 64 variables de 5 KB. La clave de servicio cabe, pero debe guardarse como secreto del Worker, nunca en el repositorio ni en el frontend.

5 Tarjeta y uso comercial: las páginas revisadas no lo dicen. Confirmar los términos de Cloudflare antes de cobrar.

Contexto del proyecto: ya existe `proxy-worker.example.js` (Worker de ejemplo para Gemini con `ALLOWED_ORIGIN`, límite por IP y tope de cuerpo). Sirve de patrón de despliegue, pero el Worker de pagos sería otro Worker, con su propio secreto.

9. Verificar aparte (no cubierto por las páginas leídas)

- Proveedor de pagos (Stripe, Wompi u otro): requisitos propios (NIT o RUT, cuenta bancaria, comisiones). Es lo que más puede cambiar el costo real.
- Clave de servicio de Firebase: crearla en Google Cloud con rol de acceso a Firestore; algunas organizaciones bloquean crear claves.
- Impuestos y facturación en Colombia.
- Por seguridad: la autorización de Pro nunca debe venir del navegador. Solo el servidor (webhook) escribe `plan`, `proStatus`, `proSince`, `proExpiresAt`, `paymentProvider`, `customerId`, `subscriptionId`.

Recomendación vigente: mantener la activación manual de Pro (editar `users/{uid}` en Firebase Console) hasta decidir el proveedor de pagos.

10. Matriz de dependencias por fase

Elemento	Solo local	Necesita Firebase	Necesita Cloudflare
Racha unificada	Sí	No	No
Escudo y rescate	Sí	No	No
XP, niveles, títulos	Sí	Opcional (sincronizar)	No
Barras «% dominado»	Sí	No	No
Insignias y misiones	Sí	Opcional	No
Moneda y tienda cosmética	Sí (frágil)	Recomendable	No
Ligas, duelos, salas, rachas compartidas	No	Sí (grupos y reglas)	No
Activar Pro por pago	No	Sí	Sí (Worker + webhook)

11. Decisiones pendientes

1. ¿Qué cuenta como «actividad válida» para la racha? (repaso, bloque, quiz; mínimos).
2. ¿Se mantiene el día de gracia automático o se reemplaza por escudos?
3. Tabla de XP: acciones y cantidades; tope diario para evitar abuso.
4. Definición de «dominado» (umbral de aciertos, número mínimo de repasos).
5. Catálogo inicial de insignias.
6. ¿Habrá moneda? Si sí, qué se compra y a qué costo.
7. Qué parte es gratis y cuál es Pro (sin ventajas de pago).
8. ¿XP local o sincronizado en Firestore? Reglas nuevas necesarias.
9. Cómo registrar los quizzes sin romper «no cambia estadísticas».
10. Aviso de logro: toast, modal, Lumi, o combinación, siempre con `aria-live`.

12. Pruebas que habrá que escribir

- Racha cuenta repasos, bloques y quizzes; no suma dos veces el mismo día.
- Eventos repetidos (doble pulsación, recarga, dos pestañas) no duplican XP.
- `hydrate` con `game` inválido, de versión más nueva o con `__proto__`.
- Exportar e importar conserva `game`; vista previa muestra sus cifras; «Borrar todo» lo vacía.
- Demo no toca datos reales.
- Perfiles examen, autodidacta y enseño no cambian su comportamiento.
- Movimiento reducido, lector de pantalla (anuncios) y traducción EN/ES de cada texto nuevo.
- Si se sincroniza: reglas de Firestore con casos permitidos y denegados, y conteo de escrituras por sesión.

13. Orden sugerido de construcción

1. Decidir los puntos de la sección 11.
2. Contenedor `state.game` + saneado + importación/copias + pruebas.
3. Actividad unificada y racha corregida (mejora del núcleo, requerida por la regla de proporción).
4. XP, niveles y títulos.
5. Barras «% dominado».
6. Insignias y misiones diarias variables.
7. Escudo de racha y rescate.
8. Moneda y tienda cosmética (si se aprueba).

9 Sincronización opcional con Firestore.

10 Funciones sociales, cuando existan grupos.

11 Pagos: Worker de Cloudflare, webhook y Bloque 7 de Firebase.

14. Fuentes citadas en la revisión de infraestructura

- Firebase pricing: https://firebase.google.com/pricing
- Firebase Authentication limits: https://firebase.google.com/docs/auth/limits
- Cloudflare Workers limits: https://developers.cloudflare.com/workers/platform/limits/
- Cloudflare Workers pricing: https://developers.cloudflare.com/workers/platform/pricing/
- Firestore REST API: https://docs.cloud.google.com/firestore/native/docs/use-rest-api